FamilyNexus – Description of Cryptographic Functionality

Draft for App Store Connect "App Encryption Documentation" and for regulatory filings. Items marked [verify] could not be confirmed from the project sources and must be checked by the publisher before submission. This document is a technical description, not legal advice.

1. Product

Product name	FamilyNexus
Version	1.0.4 (build 6)
Bundle identifier	`de.inneso.familynexus`
Platforms	iOS 15+, macOS 12+ (Android build exists, not distributed through the App Store)
Supplier / publisher	**Alexander Ziemer, Nahestr. 31, 55593 Rüdesheim, Germany**
Source code	Publicly available under AGPL-3.0-or-later: https://github.com/carpeasrael/FamilyNexus
Primary function	Family calendar, family messenger and shared shopping list
Distribution	Apple App Store (mass market, free of charge, no custom versions)

Primary function is not information security. Cryptography protects the confidentiality and integrity of the user's own messages, shared calendar entries and local data. The app is not a network infrastructure product, a VPN, a cryptographic toolkit or a security product, and it offers no user-accessible cryptographic interface (the cryptographic algorithms cannot be changed, replaced or extended by the user or by third-party code).

2. Where encryption is used

The app uses the Matrix protocol (https://spec.matrix.org) for end-to-end encrypted messaging. All cryptographic primitives are standard algorithms; no proprietary or self-designed algorithms are used.

#	Function	Algorithms	Key length	Implementation
1	Transport security to the user-chosen Matrix homeserver (HTTPS only, no plain HTTP)	TLS 1.2 / 1.3 with the cipher suites offered by the TLS library (ECDHE key exchange, AES-GCM / ChaCha20-Poly1305)	up to 256-bit symmetric	TLS stack of the Dart runtime (BoringSSL) **[verify: version and suite list]**; certificate validation against the system trust store

#	Function	Algorithms	Key length	Implementation
2	End-to-end encryption of 1:1 session setup and device-to-device messages (Olm)	Curve25519 (ECDH), Ed25519 (signatures), HKDF-SHA-256, AES-256-CBC, HMAC-SHA-256	256-bit	`vodozemac` (Rust library of the Matrix.org Foundation) via the Dart bindings `flutter_vodozemac` 0.9.0 / `vodozemac` 0.9.0 (Famedly)
3	End-to-end encryption of room messages, calendar entries and shopping list entries (Megolm)	AES-256-CBC, HMAC-SHA-256 ratchet and message authentication, Ed25519 signatures	256-bit	`vodozemac`
4	Device verification and cross-signing	Ed25519 signatures; short authentication string verification with Curve25519 ECDH, HKDF-SHA-256, HMAC-SHA-256, SHA-256	256-bit	`vodozemac`, Matrix Dart SDK 14.2.0 (Famedly, AGPL-3.0)
5	Encrypted server-side key backup and secret storage (recovery key)	Curve25519 + HKDF-SHA-256 + AES-256-CBC + HMAC-SHA-256 (backup); AES-256-CTR + HMAC-SHA-256 with HKDF-SHA-256 (secret storage); random 256-bit recovery key	256-bit	`vodozemac`, Matrix Dart SDK
6	Encrypted attachments	AES-256-CTR with random 256-bit key and 128-bit IV; SHA-256 integrity hash	256-bit	Matrix Dart SDK / `vodozemac`
7	Encrypted local database	SQLCipher 4.x: AES-256-CBC per page, HMAC-SHA-512 per page, PBKDF2-HMAC-SHA-512 (SQLCipher default iteration count)	256-bit	`sqflite_sqlcipher` 3.4.1 with SQLCipher ~> 4.6 (FMDB/SQLCipher pod); crypto provider of the pod build **[verify: CommonCrypto vs. OpenSSL]**
8	Database key	32 random bytes from a cryptographically secure random generator (`Random.secure`), stored in the operating system key store (iOS/macOS Keychain, Android Keystore)	256-bit	`flutter_secure_storage` 11.2.0 (uses the Apple Keychain – encryption built into the operating system)
9	App lock	Biometric / device credential check by the operating system (LocalAuthentication)	–	Apple operating system

Not used: key escrow or recovery by third parties, custom key exchange protocols, encryption for digital rights management, encryption of executable code, steganography, anti-tamper cryptography, quantum-key distribution, post-quantum algorithms.

3. Standards basis of the algorithms

Algorithm	Specification
AES (128/192/256)	FIPS 197
CBC, CTR modes	NIST SP 800-38A
HMAC	RFC 2104, FIPS 198-1
SHA-256 / SHA-512	FIPS 180-4
HKDF	RFC 5869
PBKDF2	RFC 8018
Curve25519 (X25519)	RFC 7748
Ed25519	RFC 8032
TLS 1.2 / 1.3	RFC 5246 / RFC 8446

The Olm and Megolm protocols are openly specified by the Matrix.org Foundation (https://gitlab.matrix.org/matrix-org/olm/-/blob/master/docs/olm.md, `megolm.md`) and combine the standard primitives above. They are not themselves published by IEEE, IETF or ITU. **[publisher to decide how to answer Apple's question about "proprietary algorithms not accepted by international standards bodies": the algorithms are standard, the protocols are open Matrix specifications.]**

4. Data flow summary

1. The user signs in to a Matrix homeserver of their choice over HTTPS.
2. Messages, shared calendar entries and shopping list entries are encrypted on the device (Megolm/Olm) before they are sent; the homeserver stores only ciphertext.
3. Private calendar entries never leave the user's own devices except over an encrypted private room of the user's own account.
4. All local data is stored in a SQLCipher-encrypted database; the database key never leaves the device key store.
5. The publisher operates no server and receives no data; there is no telemetry.

5. Export classification (publisher's assessment – to be confirmed)

Category	Mass-market software with encryption; primary function is calendar and messaging
ECCN (self-assessed)	5D992.c **[publisher to confirm]**
CCATS	none requested

Open source	The complete source code is publicly available (see §1) **[publisher to confirm whether the notification under 15 CFR §742.15(b) applies to this project]**
Countries of distribution	per App Store availability **[publisher to list, in particular whether France is included]**